



PROGRAMMING FUNDAMENTALS.

C++ I.

Create a lab report for the following programming challenge:

LAB: Number Analysis Program (Menu + Fixed Array N=10 + File Create/Load)

0) Goal

Create a menu-driven C++ program that manages a fixed-size numeric array of size **N = 10**, supports creating/loading a data file, loads up to N values into the array, and provides analysis operations (min, max, total, average) on the loaded data.

1) Global Program Constraints

- 1.1 The program must use a fixed-size array with size **N = 10**.
 - 1.2 The program must not use dynamic memory allocation for the dataset (no `new`, no resizing containers for storage).
 - 1.3 The program must track how many valid numbers are currently stored in the array.
 - 1.4 The program must enforce that analysis operations are only allowed when the array contains at least one valid number.
-

2) Student File Format Requirement

- 2.1 Any data file used by the program must be a plain text file.
- 2.2 The file must contain numeric values, with **one number per line**.
- 2.3 The file may contain fewer than N numbers but must not be assumed to contain exactly N.
- 2.4 If the file contains more than N numbers, the program must not store more than N values in the array.

3) Functional Requirements

3.1 Menu System

- 3.1.1 The program must repeatedly display a menu until the user selects Exit.
- 3.1.2 The menu must include, at minimum, the following options:
 - 3.1.2.1 Create a data file (user enters up to N numbers and the program saves them)
 - 3.1.2.2 Load numbers from a file into the array (up to N numbers)
 - 3.1.2.3 Display the lowest number (analysis)
 - 3.1.2.4 Display the highest number (analysis)
 - 3.1.2.5 Display the total of the numbers (analysis)
 - 3.1.2.6 Display the average of the numbers (analysis)
 - 3.1.2.7 Exit
- 3.1.3 After completing any option other than Exit, the program must return to the menu.

3.2 Create File Option

- 3.2.1 When the user selects “Create a data file,” the program must prompt for a file name to create/overwrite.
- 3.2.2 The program must allow the user to enter numeric values one at a time, up to a maximum of **N = 10** values.
- 3.2.3 The program must save each accepted numeric value to the file, with one value per line.
- 3.2.4 The program must stop accepting values when either:
 - 3.2.4.1 The user has entered N valid numbers, or
 - 3.2.4.2 The user chooses to stop early (course policy: the program must provide a controlled way to stop early).
- 3.2.5 After file creation, the program must report how many numbers were saved.

3.3 Load From File Option

- 3.3.1 When the user selects “Load numbers from a file,” the program must prompt for the input file name.
- 3.3.2 The program must read numeric values from the file and store them into the fixed array.
- 3.3.3 The program must load no more than **N = 10** valid values into the array.
- 3.3.4 After loading, the program must report how many values are currently stored in the array.
- 3.3.5 Loading a new file must replace the previously stored dataset (array contents and count must reflect the newly loaded data only).

3.4 Analysis Operations

- 3.4.1 Lowest Value: display the lowest number among the values currently stored in the array.
 - 3.4.2 Highest Value: display the highest number among the values currently stored in the array.
 - 3.4.3 Total: display the sum of the values currently stored in the array.
 - 3.4.4 Average: display the arithmetic mean of the values currently stored in the array.
-

3.5 Output Requirements

- 3.5.1 All results must be clearly labeled.
 - 3.5.2 All results must be computed from the array values currently loaded.
 - 3.5.3 The program must not hard-code results.
-

4) Modularity Requirements (Mandatory)

The program must be modular. At minimum, separate functions must exist for:

- 4.1 Displaying the menu and obtaining the user's choice.
- 4.2 Creating/writing the data file (prompting and saving up to N values).
- 4.3 Loading values from a file into the array (up to N values).
- 4.4 Finding the lowest value using the current count of loaded elements.
- 4.5 Finding the highest value using the current count of loaded elements.
- 4.6 Computing the total using the current count of loaded elements.
- 4.7 Computing the average using the current count of loaded elements.
- 4.8 Displaying results (or formatting output consistently).

`main` must coordinate flow and must not contain all logic.

5) Input Validation Requirements (Where Validation Must Occur)

5.1 Menu Choice Validation

- 5.1.1 Validate that the menu entry is numeric.
- 5.1.2 Validate that the choice is within the menu option range.
- 5.1.3 On invalid choice, display an error and re-display the menu without terminating.

5.2 File Name Validation (Both Create and Load)

- 5.2.1 Validate that the filename input is not empty.
 - 5.2.2 For loading: validate that the file opens successfully for reading.
 - 5.2.3 For creating: validate that the file opens successfully for writing.
 - 5.2.4 On open failure, display an error and return to the menu safely.
-

5.3 Numeric Entry Validation (Create File Option)

- 5.3.1 Validate each entered value is numeric before accepting it as a dataset value.
 - 5.3.2 Reject invalid numeric input safely and allow re-entry without crashing.
 - 5.3.3 Ensure the program never stores more than N values.
-

5.4 File Content Validation (Load From File Option)

- 5.4.1 Validate that each value read from the file is numeric.
 - 5.4.2 If invalid data is found, the program must respond safely according to course policy (must not crash).
 - 5.4.3 Ensure the program never stores more than N valid values.
-

5.5 Data Availability Validation (Before Analysis)

- 5.5.1 If the array contains zero valid numbers, analysis options must not run.
 - 5.5.2 The program must notify the user that they must load data first (via Create File and then Load, or Load directly).
-

5.6 Mathematical Validation

- 5.6.1 Prevent division by zero when computing the average.
 - 5.6.2 Prevent min/max computations when the count is zero.
-

6) Program Behavior Requirements

- 6.1 The program must loop until Exit is chosen.
 - 6.2 The program must exit cleanly and release all resources.
 - 6.3 The program must not terminate unexpectedly due to invalid input or file errors.
-

7) Deliverables

- 7.1 C++ source code file(s).
 - 7.2 At least one sample data file created using the program's "Create a data file" option.
-

If you want, I can also produce a **grading rubric mapped 1:1** to these numbered requirements (so you can check off "3.2.3 satisfied," etc.).

Submission

- 1. Analysis.cpp. (One file: Libraries, Prototypes, main(), function definitions)
 - 2. PDF lab report. (Demonstrate that all requirements are met)
-

Any question? Just email me.
LDV.